

MAP ASSIGNMENT - 3

1.Event Validation on isNotNull,isEmail,isNumber

Project: ValidationRules

Objective: We want to validate the request payload in this order:

1. Is Not Null → payload.name
2. Is Email → payload.email
3. Is Number (with range 0–100) → payload.marks

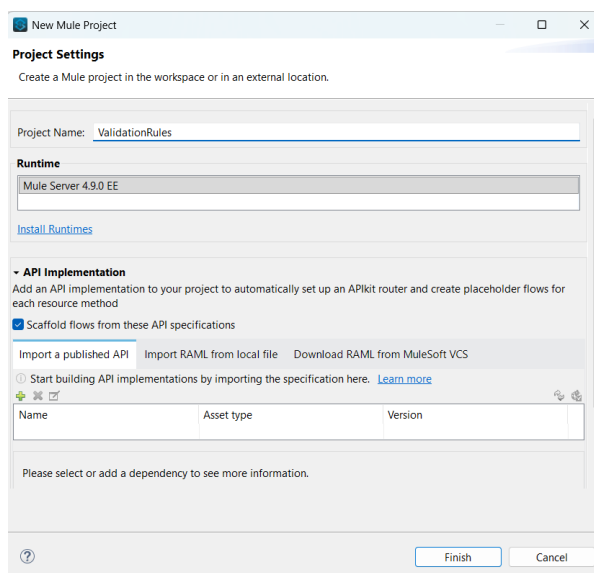
Finally return the payload using a Transform Message.

Step 1: Create a New Project

Open Anypoint Studio

Go to File → New → Mule Project

Project Name: Validationrules



Step 2: Configure HTTP Listener

Drag an HTTP Listener into the flow.

Create a new HTTP Listener Configuration:

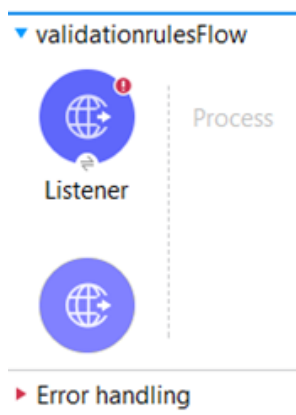
Host: 0.0.0.0

Port: 8081

In the Listener component:

Path: /validation

Purpose: This will accept incoming POST requests.



Listener x Console Problems

General

MIME Type

Redelivery

Responses

Advanced

Metadata

Notes

Help

There are no errors.

Display Name: Listener

Basic Settings

Connector configuration: HTTP_Listener_config

General

Path: /notnull

Step 3: Add Validation – Is Not Null

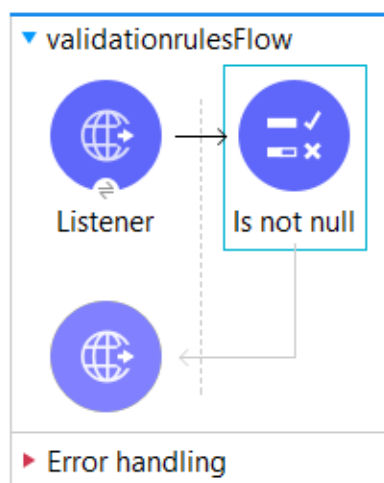
Drag Validation → Is not null after the Listener.

Configure:

Value: #[payload.name]

Message: Name should not be null

Purpose: Ensures the request contains a name value.



Is not null x Console Problems

General

Error Mapping

Metadata

Notes

Help

There are no errors.

Display Name: Is not null

Basic Settings

Module configuration:

General

Value: fx payload.name

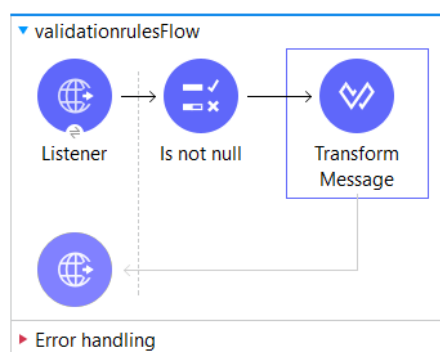
Error options

Message: fx Name cannot be null

Step 4: Add Transform Message

Drag a Transform Message as the last step.

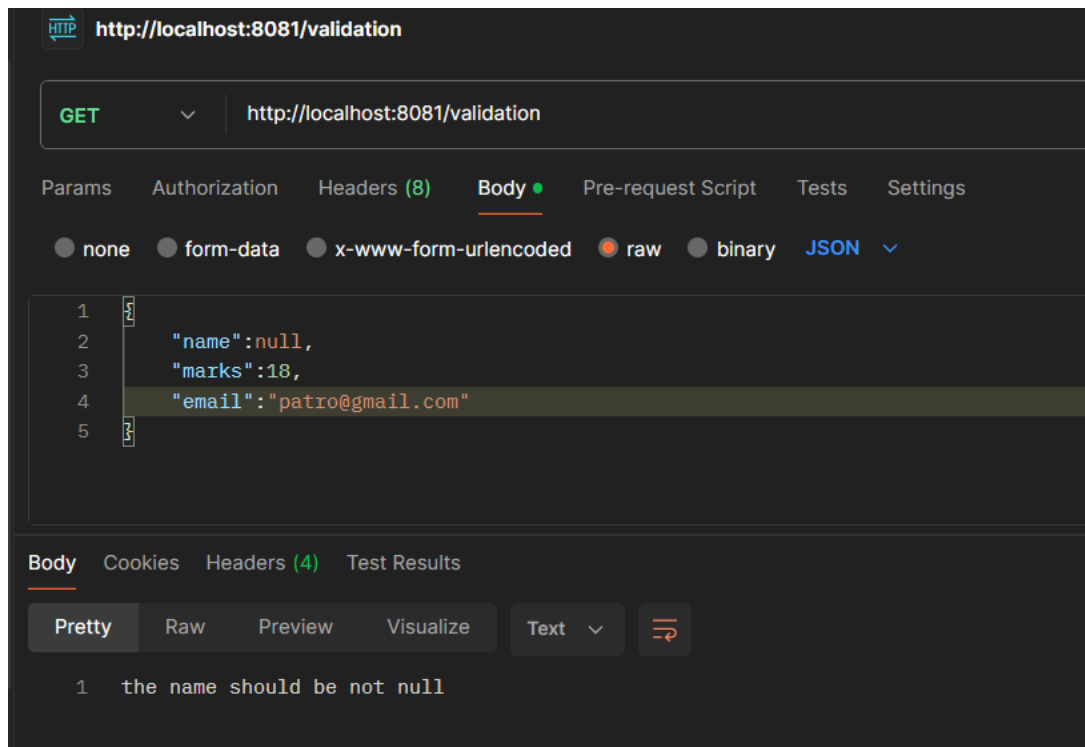
Enter the following DWL code:



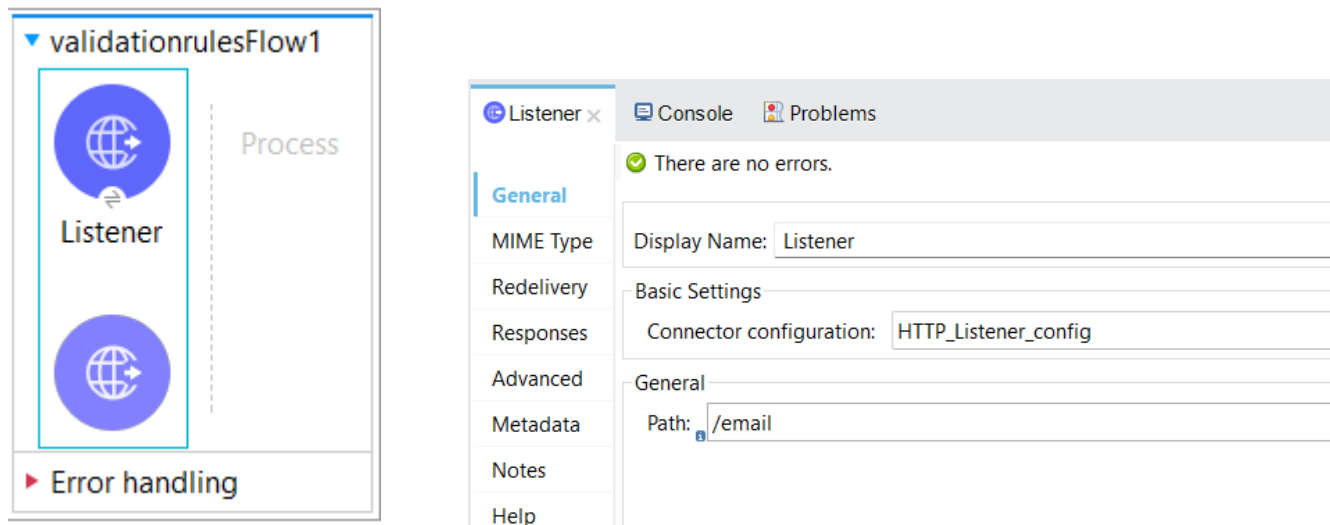
Output Payload

```
1 %dw 2.0
2 output application/json
3 ---
4 payload
```

Output:



Step 5: Repeat Step 2 again



Step 6: Add Validation – Is Email

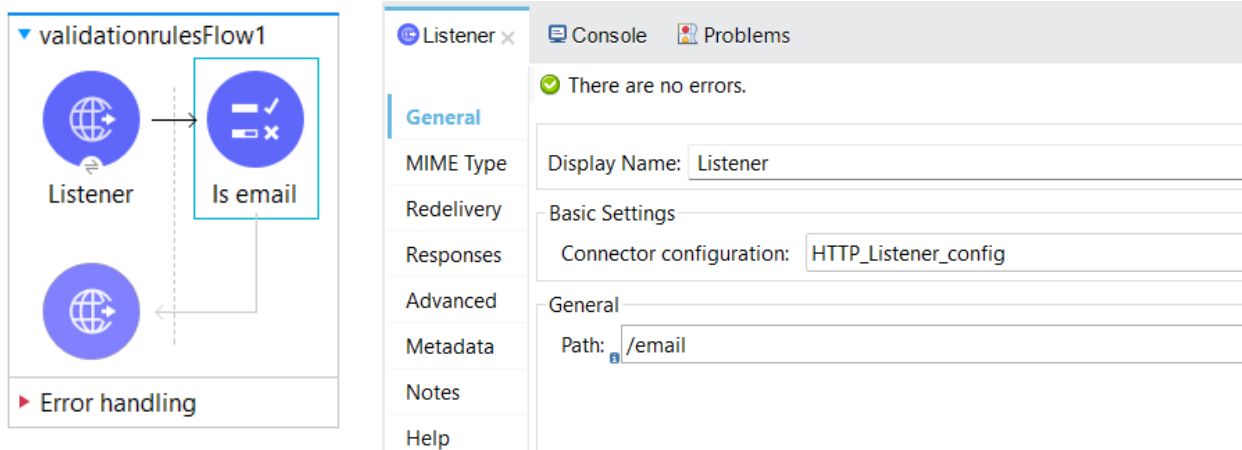
Drag Validation → Is email

Configure:

Value: `#[payload.email]`

Message: Please enter valid email

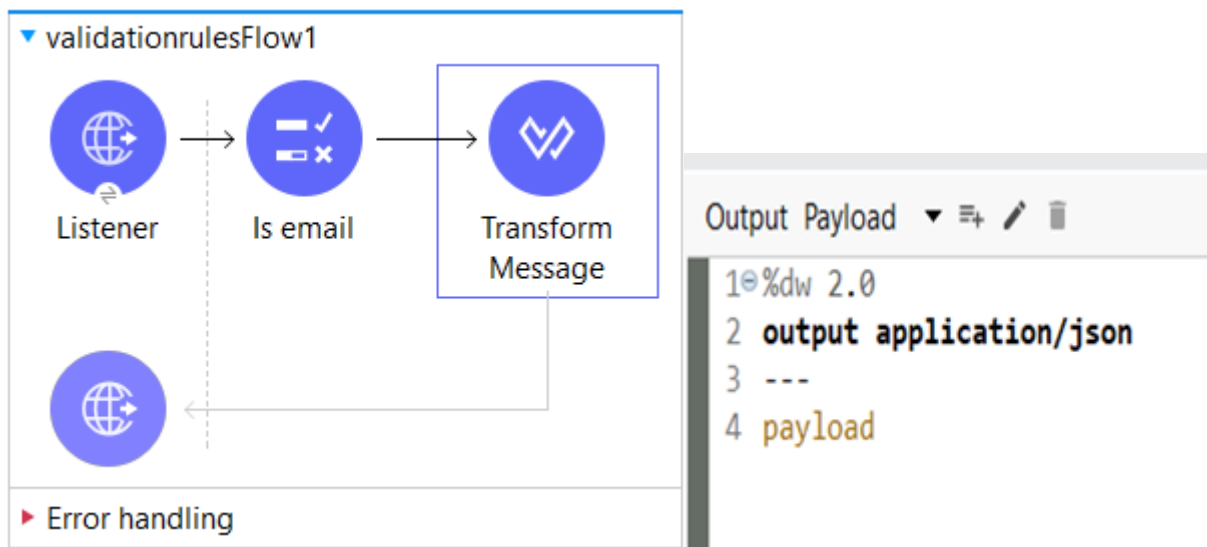
Purpose: Ensures the request contains a valid email format.



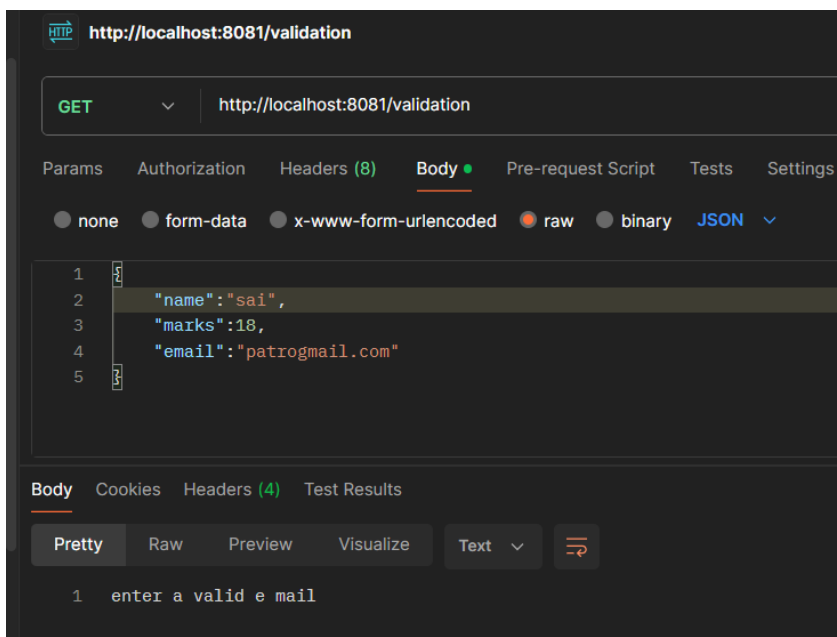
Step 7: Add Transform Message

Drag a Transform Message as the last step.

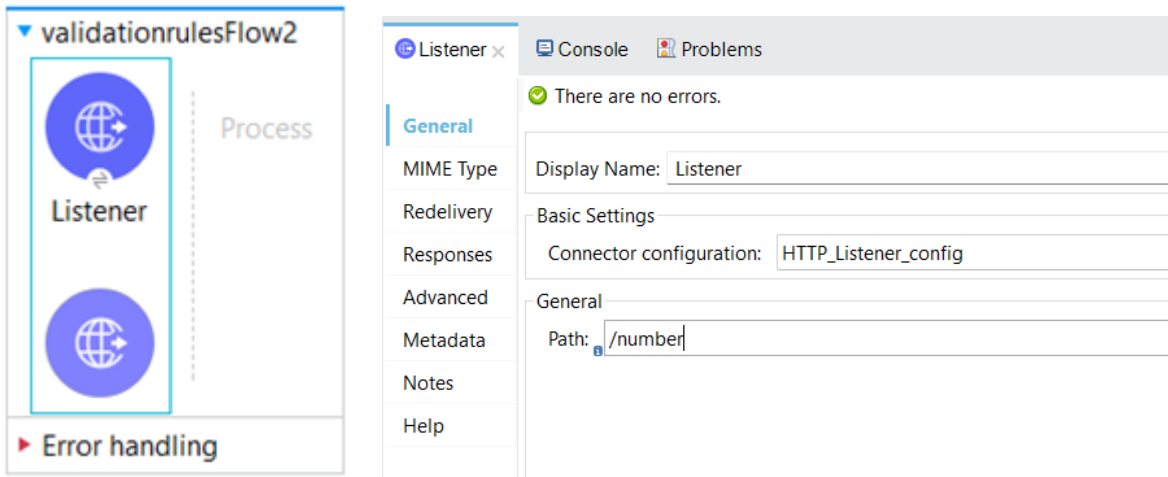
Enter the following DWL code:



Output:



Step8:Repeat Step 2 again



The screenshot shows the 'validationrulesFlow2' process diagram on the left, which contains a 'Listener' node and an 'Error handling' section. On the right, the 'Listener' configuration panel is open, showing the 'General' tab. The 'Display Name' is 'Listener', the 'Connector configuration' is 'HTTP_Listener_config', and the 'Path' is '/number'.

Step9:Add Validation – Is Number

Drag Validation → Is number Listener.

Configure:

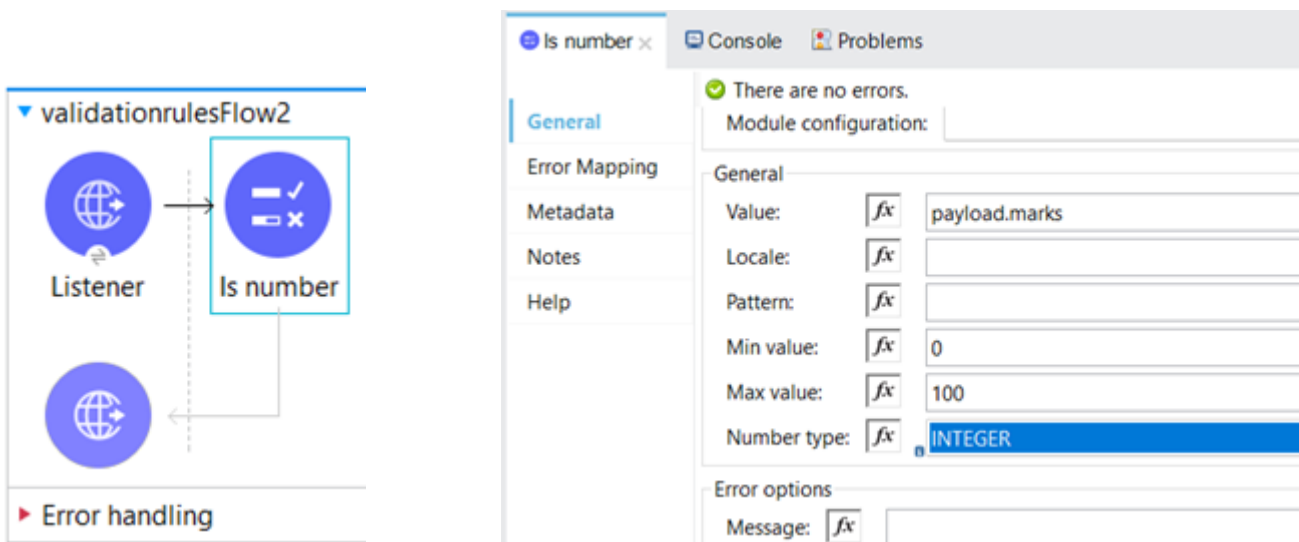
Value: #[payload.marks]

Message: Please enter valid marks

Minimum Value: 0

Maximum Value: 100

Purpose: Ensures marks is a number between 0 and 100.

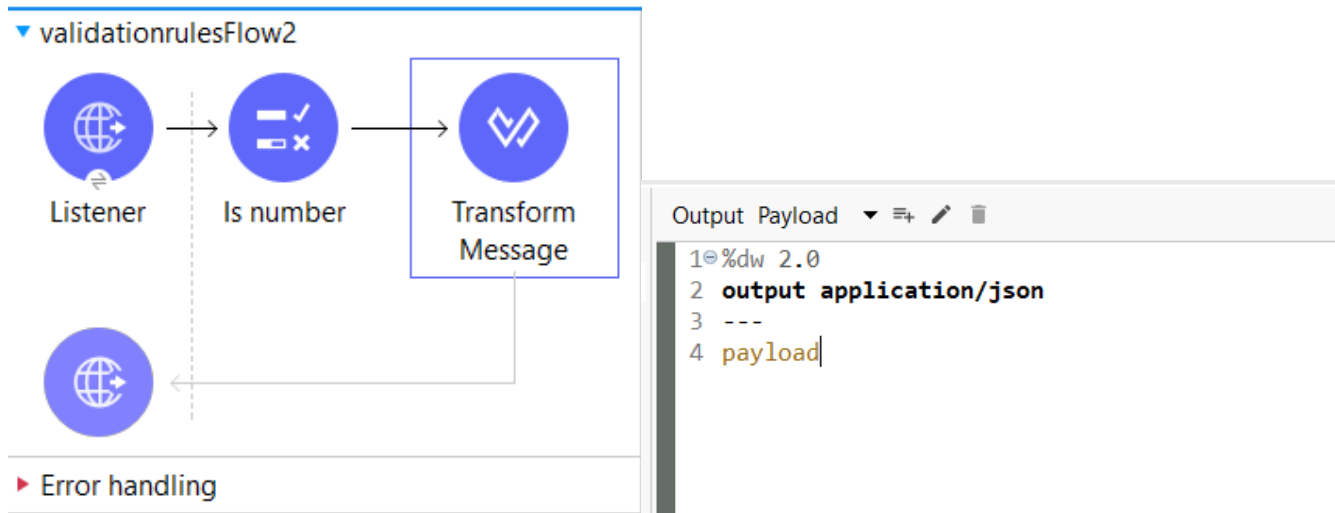


The screenshot shows the 'validationrulesFlow2' process diagram on the left, which now includes a 'Listener' node, an 'Is number' validation node, and an 'Error handling' section. On the right, the 'Is number' configuration panel is open, showing the 'General' tab. The 'Value' is 'payload.marks', the 'Locale' is empty, the 'Pattern' is empty, the 'Min value' is '0', the 'Max value' is '100', and the 'Number type' is 'INTEGER'. The 'Error options' section shows the 'Message' is empty.

Step 10: Add Transform Message

Drag a Transform Message as the last step.

Enter the following DWL code:



Output:

HTTP http://localhost:8081/validation

GET http://localhost:8081/validation

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON ▼

```
1 {
2   "name": "sai",
3   "marks": 188,
4   "email": "patro@gmail.com"
5 }
```

Body Cookies Headers (4) Test Results

Pretty Raw Preview Visualize Text ▼

```
1 enter marks between 0-100
```

2.Choice Router

Project: StudentGrades

Objective: We want to accept a student's marks from the request payload, set it as a variable, and then use Choice Router to assign grades:

$\geq 90 \rightarrow \text{Grade} = \text{A}$

$\geq 80 \rightarrow \text{Grade} = \text{B}$

$\geq 70 \rightarrow \text{Grade} = \text{C}$

$\geq 60 \rightarrow \text{Grade} = \text{D}$

$\geq 50 \rightarrow \text{Grade} = \text{E}$

$\geq 40 \rightarrow \text{Grade} = \text{Pass}$

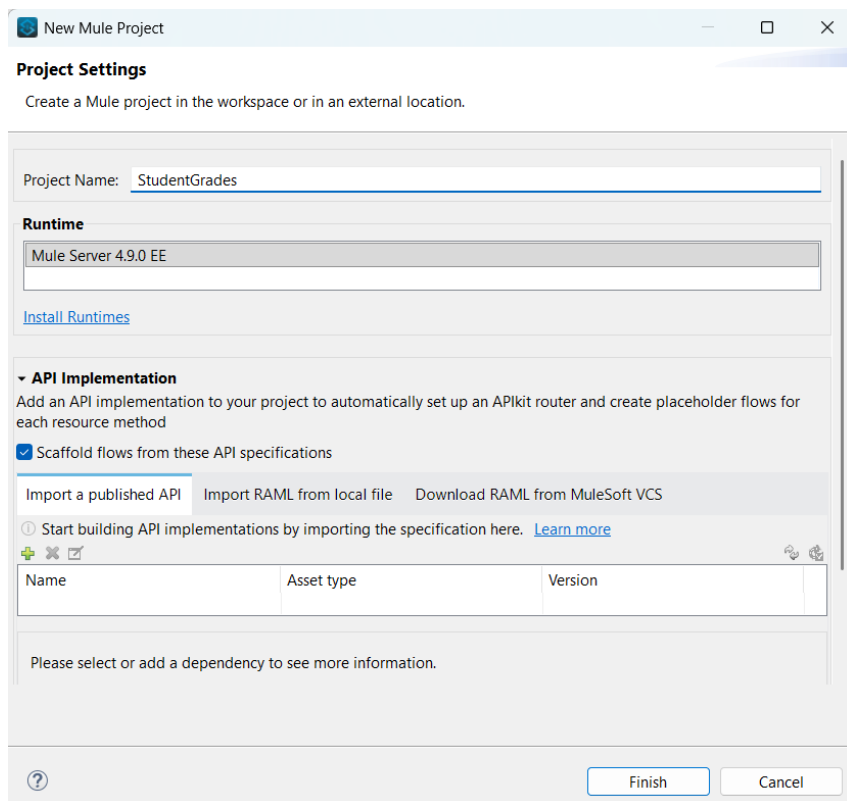
$< 40 \rightarrow \text{Fail}$

Step 1: Create New Project

Open Anypoint Studio

File $\rightarrow$ New Mule Project

Name: StudentGrades



Step 2: Add HTTP Listener

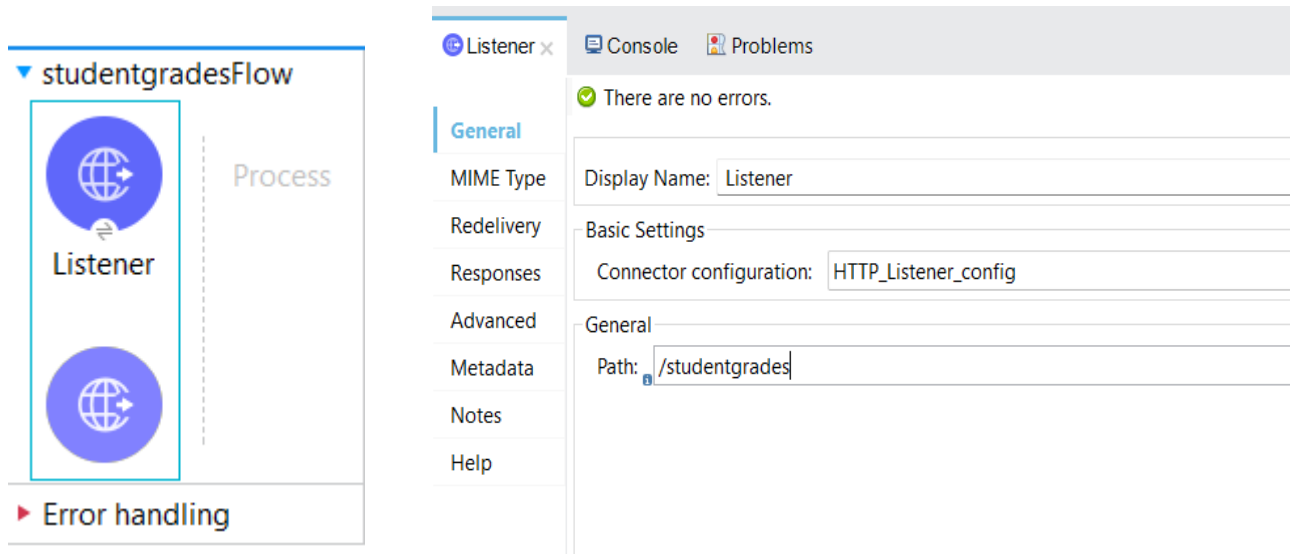
Drag an HTTP Listener into the flow.

Configure:

Path = /grades

Port = 8081

Purpose: Accepts incoming request with student marks



The screenshot shows the 'studentgradesFlow' process in a flow editor. The 'Listener' component is highlighted with a red box. To the right, the configuration panel for the 'Listener' component is visible. The 'General' tab is selected, showing the 'Display Name' as 'Listener', 'Basic Settings' with 'Connector configuration' set to 'HTTP_Listener_config', and 'General' settings with 'Path' set to '/studentgrades'. The 'Console' tab shows 'There are no errors.'

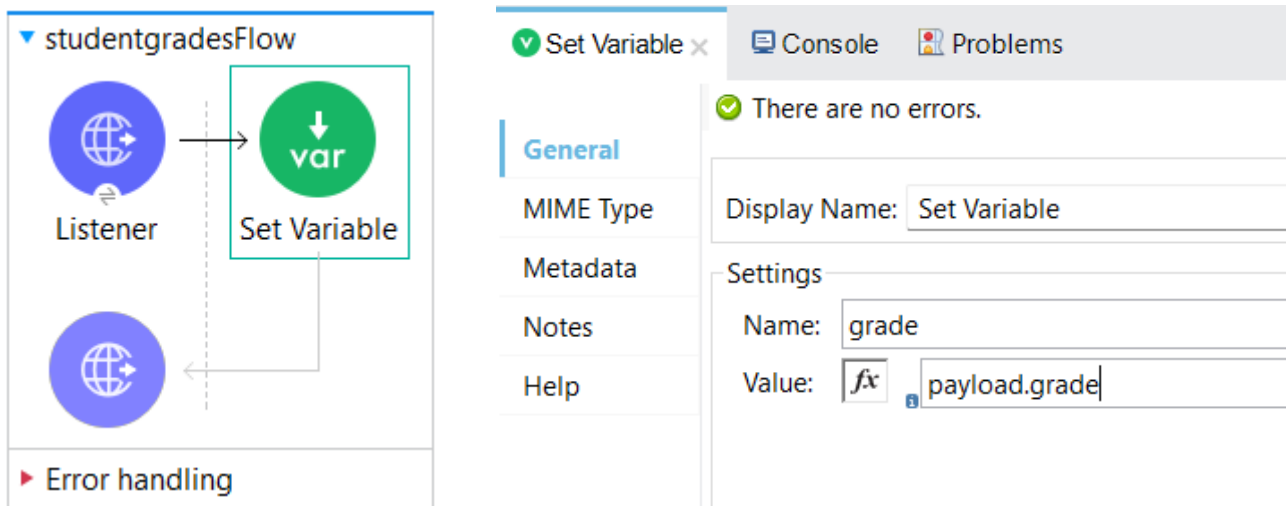
Step 3: Set Variable

Drag a Set Variable component after Listener.

Name = grade

Value = #[payload.grade]

Purpose: Store the marks from input payload into a variable.



The screenshot shows the 'studentgradesFlow' process in a flow editor. The 'Set Variable' component is highlighted with a red box. To the right, the configuration panel for the 'Set Variable' component is visible. The 'General' tab is selected, showing the 'Display Name' as 'Set Variable', 'Settings' with 'Name' set to 'grade' and 'Value' set to '#[payload.grade]'. The 'Console' tab shows 'There are no errors.'

Step 4: Choice Router

Drag a Choice component after Set Variable.

Add multiple When conditions and a Default.

Conditions (DW expressions):

#[vars.grade >= 90] → Transform → "Grade: A"

#[vars.grade >= 80] → Transform → "Grade: B"

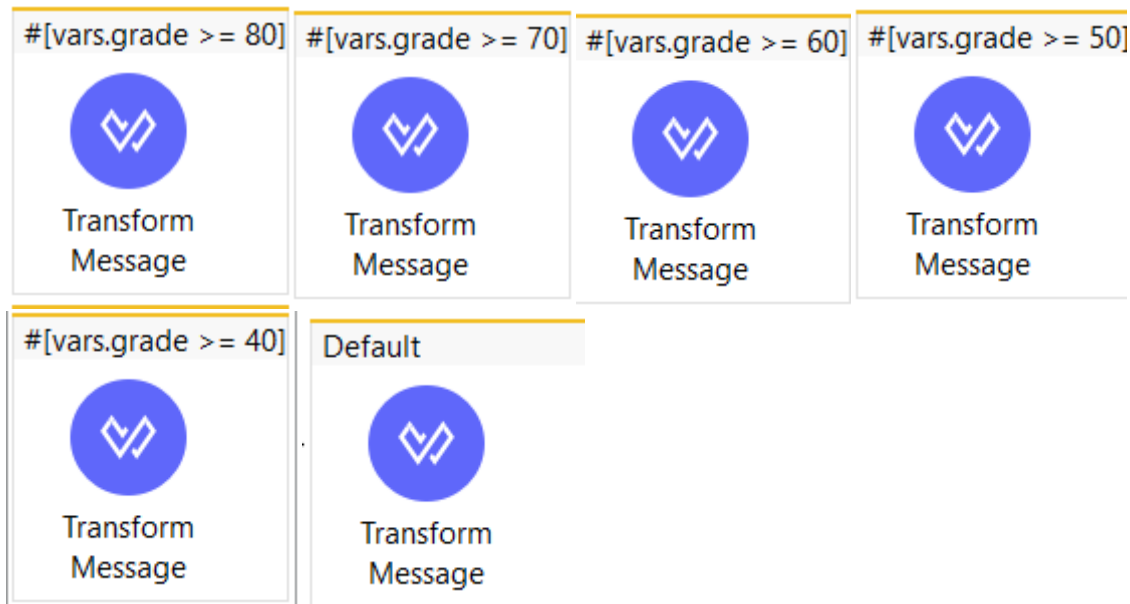
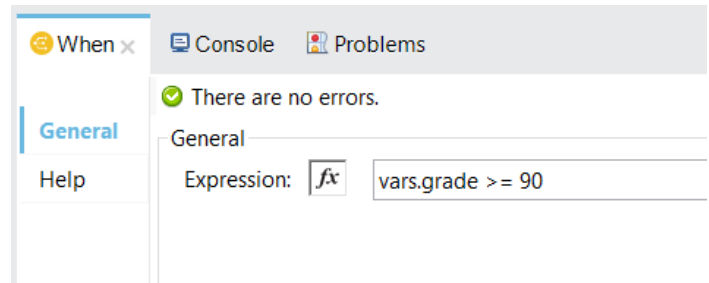
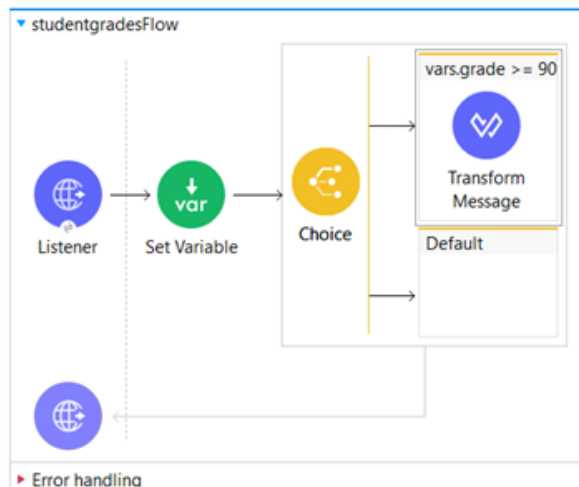
#[vars.grade >= 70] → Transform → "Grade: C"

`#[vars.grade >= 60] → Transform → "Grade: D"`

`#[vars.grade >= 50] → Transform → "Grade: E"`

`#[vars.grade >= 40] → Transform → "Pass"`

Default → Transform → "Fail"



Step 5: Transform Message for Each Case

Inside each branch, add a Transform Message with something like:

(Modify "Grade: A" for each branch accordingly.)

Step 6: Run and Test

Run the project → Mule Application

Test with Postman:

GET

▼

http://localhost:8081/studentgrades

Params

Authorization

Headers (9)

Body ●

Scripts

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

1

{

2

|

"grade": 95

3

}

4

Body

Cookies

Headers (3)

Test Results

🕒

{}

JSON

▼

▶

Preview

🖼️

Visualization

1

"Grade: A"